BACKEND ENGINEERING ROADMAP

02

Intermediate

Scalable Services and Real-Time Systems

DURATION
16 weeks

WEEKLY EFFORT
5-10 hours per week

CAPSTONE
Real-Time Chat and E-commerce API

PROFESSIONAL ENGLISH EDITION

LEVEL 02 / ORIENTATION

Purpose and expectations

Move from a single reliable service to systems that coordinate real-time clients, multiple data stores, caches, reverse proxies, and deployment automation. The emphasis is choosing the right communication and storage pattern for each workload.

RECOMMENDED DURATION

16 weeks

WEEKLY COMMITMENT

5-10 hours per week

ENTRY CRITERIA

Level 1 or equivalent production API experience

PRIMARY PROJECT

Real-Time Chat and E-commerce API

CORE FOCUS

Real-time communication, caching, NoSQL, clean architecture, edge delivery, and scalability

How to use this roadmap

1. Study for understanding, then validate the idea through implementation.
2. Keep evidence: source code, tests, measurements, diagrams, and short decision records.
3. Prefer a working, explained trade-off over a large but unfinished feature set.
4. Complete each assessment from a clean checkout before advancing.

Capabilities at completion

- Choose between WebSockets and Server-Sent Events and scale real-time delivery across service instances.
- Apply Redis cache-aside, TTL, invalidation, pub/sub, atomic counters, rate limiting, and stampede protection.
- Design MongoDB documents and aggregation pipelines with evidence-based indexes.
- Configure Nginx for TLS termination, reverse proxying, load balancing, compression, caching, CORS, and WebSocket upgrades.
- Use HTTP validators and cache directives appropriately for static, dynamic, and sensitive resources.
- Organize domain, application, infrastructure, and composition layers with inward dependency direction.
- Apply Repository, Factory, Adapter, and Observer patterns when they clarify boundaries.
- Write integration tests against real infrastructure and add them to a multi-version CI pipeline.
- Implement idempotency for critical writes and backpressure for overloaded consumers.
- Use BFS, DFS, stacks, queues, trees, and hash tables in practical problem solving.

LEVEL 02 / PLAN

Roadmap at a glance

PHASE	SCHEDULE	FOCUS	EVIDENCE
Real-time and cache	Weeks 1-3	WebSockets, SSE, Redis	URL Shortener
Data and edge	Weeks 4-6	MongoDB, Nginx, Clean Architecture	Production delivery layer
Real-time capstone	Weeks 7-10	Chat, integration tests, CI/CD	Scalable Chat service
Algorithms and commerce	Weeks 11-14	Data structures, transactions, idempotency	E-commerce API
Review	Weeks 15-16	Refactoring and assessment	Portfolio demonstration

Completion rule

Do not treat elapsed time as completion. Advance when you can demonstrate the deliverables, explain the design decisions, and reproduce the result.

LEVEL 02 / EXECUTION

Weekly learning plan

WEEK 01 WebSockets and Server-Sent Events

4h study + 5h practice

LEARNING FOCUS

- WebSocket handshake, rooms, namespaces, heartbeats, and reconnect behavior.
- SSE over `text/event-stream`, automatic reconnection, and `Last-Event-ID`.
- Bidirectional vs server-to-client requirements.
- Sticky sessions and cross-instance adapters.

APPLIED WORK

- Build an SSE endpoint and browser client.
- Implement Socket.IO join, leave, message, and typing events.
- Test three concurrent rooms and direct socket-to-socket messaging.
- Document the decision boundary between WebSocket, SSE, and polling.

SSE ENDPOINT

```
app.get('/api/events/stream', (req, res) => {
  res.writeHead(200, {
    'Content-Type': 'text/event-stream',
    'Cache-Control': 'no-cache',
    Connection: 'keep-alive',
  });
  const timer = setInterval(() => {
    res.write(`data: ${JSON.stringify({ timestamp: Date.now() })}\n\n`);
  }, 5000);
  req.on('close', () => clearInterval(timer));
});
```

DELIVERABLE A real-time prototype with three rooms, direct messages, reconnection behavior, and a short protocol decision note.

WEEK 02 Redis Cache Patterns and Pub/Sub

3h study + 5h practice

LEARNING FOCUS

- Cache-aside, write-through, write-behind, TTL, and invalidation.
- Cache stampede causes and mitigation.
- Pub/sub for cross-instance events and its delivery limitations.
- Atomic counters and fixed-window rate limiting.

APPLIED WORK

- Implement a generic cache-aside helper.
- Invalidate product keys after writes.
- Bridge Redis pub/sub to local WebSocket connections.
- Apply route-specific IP limits.

CACHE-ASIDE PATTERN

```
async function getOrSet<T>(key: string, load: () => Promise<T>, ttl = 60) {
  const cached = await redis.get(key);
  if (cached) return JSON.parse(cached) as T;
  const value = await load();
  await redis.set(key, JSON.stringify(value), 'EX', ttl);
  return value;
}
```

DELIVERABLE Rate limiting by IP and route, plus a tested cache-aside implementation with explicit invalidation.

WEEK 03

Mini Project: URL Shortener

Use caching, atomic operations, redirects, and containerization in a compact service.

LEARNING FOCUS

- Base62 identifiers and collision handling.
- Permanent vs temporary redirects.
- Bidirectional cache keys and atomic click counters.

APPLIED WORK

- Implement `POST /shorten`, `GET /:id`, and `GET /:id/stats`.
- Validate URLs and reject unsafe schemes.
- Add Redis TTL, `INCR`, Docker Compose, and tests.

DELIVERABLE A containerized URL Shortener with unique seven-character IDs, redirects, statistics, validation, and automated tests.

WEEK 04

MongoDB Modeling and Aggregation

3h study + 5h practice

LEARNING FOCUS

- Embedded vs referenced documents and workload-driven schema design.
- Compound indexes for chat history access patterns.
- Aggregation stages including `$match`, `$sort`, `$limit`, `$lookup`, `$unwind`, and `$project`.
- When PostgreSQL remains the better choice.

APPLIED WORK

- Model users, rooms, memberships, and messages.
- Create a compound `{ roomId: 1, createdAt: -1 }` index.
- Build three useful aggregation pipelines and inspect their plans.

DELIVERABLE A chat data model, index rationale, and three aggregation pipelines with representative test data.

WEEK 05

Nginx, HTTP Caching, Compression, CORS, and TLS

4h study + 5h practice

LEARNING FOCUS

- Reverse proxy vs load balancer responsibilities.
- `Cache-Control`, ETag, `Last-Modified`, conditional requests, and `304` responses.
- Brotli vs gzip compatibility and CPU-bandwidth trade-offs.
- CORS preflight rules and least-privilege origins.
- WebSocket upgrades, TLS termination, and route-level rate limiting.

APPLIED WORK

- Proxy two application instances.
- Enable TLS and HTTP/2 where supported.
- Configure compression and immutable caching for fingerprinted assets.
- Keep sensitive responses at `no-store` and validate headers with `curl`.

DELIVERABLE A reviewed Nginx configuration covering TLS, proxying, WebSockets, compression, caching, CORS, and 30 requests-per-second rate limiting.

WEEK 06 Clean Architecture in Depth

5h study + 5h practice

LEARNING FOCUS

- Entities and business rules in the domain layer.
- Use cases and ports in the application layer.
- Database, HTTP, and third-party adapters in infrastructure.
- Dependency injection in the composition root.
- Pattern selection based on pressure, not ceremony.

APPLIED WORK

- Migrate the Level 1 Task Manager.
- Extract repository and email interfaces.
- Make domain tests run without frameworks or infrastructure.
- Verify that dependencies point inward.

ARCHITECTURE AND SCOPE

- Domain never imports infrastructure
- Application owns use-case contracts
- Infrastructure implements ports
- Main composes concrete dependencies

DELIVERABLE A cleanly layered Task Manager with dependency boundaries demonstrated by isolated domain tests.

WEEKS 07-08 Capstone: Real-Time Chat

Build and scale a stateful real-time product while keeping business rules testable.

LEARNING FOCUS

- Room lifecycle, messaging, history, and online presence.
- MongoDB persistence and cursor pagination.
- Redis adapter and presence TTL across multiple instances.
- Nginx affinity, TLS, uploads, and authentication.

APPLIED WORK

- Run two application instances behind Nginx.
- Implement JWT authentication and dynamic rooms.
- Store history and paginate it safely.
- Upload images with bounded size and type validation.
- Package app, MongoDB, Redis, and Nginx in Compose.

DELIVERABLE A complete real-time chat repository with multi-instance verification, architecture documentation, tests, and a reproducible environment.

WEEK 09 Integration Testing

2h study + 5h practice

LEARNING FOCUS

- When integration tests provide more value than mocks.
- Isolated infrastructure with Testcontainers.
- Lifecycle management, deterministic data, and timeouts.

APPLIED WORK

- Start MongoDB in the test suite.
- Test save, history retrieval, and room listing.
- Ensure containers and connections are always released.

DELIVERABLE Three reliable chat integration tests that run locally and in CI without shared state.

WEEK 10 Advanced CI/CD and Deployment

2h study + 5h practice

LEARNING FOCUS

- Matrix builds, service containers, caches, artifacts, and immutable image tags.
- Deployment gates and secret handling.
- Rollback-friendly delivery.

APPLIED WORK

- Test supported Node.js versions.
- Run MongoDB and Redis in CI.
- Build and publish an image tagged with the commit SHA.
- Deploy only after the test job succeeds.

DELIVERABLE A multi-version pipeline that tests, publishes an immutable image, and executes a documented deployment step.

WEEK 11 Algorithms and Data Structures

4h study + 5h practice

LEARNING FOCUS

- Stacks, queues, binary trees, graphs, hash tables, tries, and priority queues.
- Breadth-first and depth-first traversal.
- Dijkstra, greedy reasoning, and traversal complexity.

APPLIED WORK

- Implement stack, queue, tree node, BFS, DFS, and BST validation from scratch.
- Solve eight easy or medium problems.
- Document approach, complexity, and alternative solutions.

DELIVERABLE Eight reviewed solutions plus reusable implementations of the core structures and traversals.

WEEKS 12-14 Capstone: E-commerce API

Combine transactions, cache invalidation, authorization, resilience, and delivery concerns in a business-critical API.

LEARNING FOCUS

- Product and category APIs, roles, carts, and transactional checkout.
- Cache-aside with invalidation for popular products and categories.
- Idempotency keys for safe retries.
- Bounded queues, rejection, stream backpressure, and circuit breakers.

APPLIED WORK

- Use PostgreSQL transactions for stock and order updates.
- Reserve idempotency keys atomically with Redis SET NX and TTL.
- Return the original response for repeated keys.
- Add Nginx, Docker Compose, OpenAPI, and a full checkout integration test.

ARCHITECTURE AND SCOPE

- Node.js/TypeScript API
- PostgreSQL system of record
- Redis cache and idempotency store
- Nginx edge proxy
- Clean Architecture and repository ports

DELIVERABLE A documented E-commerce API with ACID checkout, role-based access, cache invalidation, idempotent retries, integration tests, and deployable infrastructure.

WEEK 15 Refactoring and Interview Practice

LEARNING FOCUS

- Architecture consistency across the E-commerce codebase.
- Test gaps at critical boundaries.
- Medium-level algorithm practice.

APPLIED WORK

- Complete any missing Clean Architecture boundaries.
- Add high-value integration tests.
- Solve five medium interview problems and review them aloud.

DELIVERABLE A release candidate capstone and a concise interview-practice log.

WEEK 16 Level Assessment

LEARNING FOCUS

- Scale WebSockets and choose SSE appropriately.
- Explain cache-aside, stampedes, MongoDB modeling, Nginx roles, HTTP caching, compression, CORS, and Clean Architecture.

APPLIED WORK

- Answer at least nine of twelve questions accurately.
- Demonstrate both Chat and E-commerce projects from clean checkouts.

- Defend idempotency, implement BFS and DFS, and design a four-stage CI/CD pipeline.
- Explain one scalability bottleneck and the evidence used to address it.

DELIVERABLE Pass when the assessment threshold is met and both capstones are complete, tested, and available in GitHub.

LEVEL 02 / REFERENCE

Recommended source material

Real-time systems

Socket.IO documentation; MDN Server-Sent Events; Redis documentation and patterns.

Data and delivery

MongoDB schema design and aggregation documentation; Nginx reverse proxy and caching guidance.

Architecture and tests

Clean Architecture references; Refactoring Guru; Testcontainers for Node.js.

Resilience

HTTP caching specifications; idempotency patterns; Node.js stream and backpressure guidance.

Completion perspective

Level 2 is complete when you can make clear trade-offs among protocols, stores, caches, and delivery patterns, then prove the system works across multiple instances.